

РО 1.3. Базовое администрирование Linux

Лекция 6. Процессы, потоки ввода/вывода, pipeline и управление заданиями

Цель лекции

Научиться понимать процессы Linux, сигналы, стандартные потоки, перенаправление и простые конвейеры команд.

Список учебных вопросов

1. Процесс в Linux с практической точки зрения администратора
2. PID, PPID, состояние процесса, владелец процесса
3. Просмотр процессов: ps, top, htop на уровне назначения
4. Сигналы процессам: SIGTERM, SIGKILL, SIGHUP
5. Завершение процессов: kill, pkill, killall
6. Стандартные потоки: stdin, stdout, stderr
7. Перенаправление вывода и ошибок: >, >>, 2>, 2>&1
8. Конвейер pipeline: передача вывода одной команды на вход другой
9. Базовые инструменты обработки вывода: grep, cut, sort, uniq, wc

1. Процесс в Linux

Процесс — запущенная программа со своим PID, памятью, владельцем и состоянием.

Для администратора процесс важен, потому что службы, shell и пользовательские программы работают как процессы.

Если сервер тормозит, зависает или служба не отвечает, диагностика часто начинается с процессов.

Нельзя завершать процесс, не понимая его роль.

Процесс и программа

Программа — файл на диске, например
/usr/bin/nginx

Процесс — выполняющийся экземпляр программы.

Одна программа может иметь много процессов.

Завершение процесса не удаляет программу с диска.



ПРОЦЕСС И ПРОГРАММА

Ключевые концепции в Linux



Программа (file) — это пассивный код на диске.
Процесс — это запущенная программа в памяти.

1. ПРОГРАММА

Программа (исполняемый файл) — это набор инструкций, хранящийся на диске.



/usr/bin/vim

- ✓ Пассивна (ничего не делает сама по себе)
- ✓ Хранится в виде файла на диске
- ✓ Может быть скопирована, перемещена, передана другим пользователям
- ✓ При запуске становится процессом

Примеры программ:

/bin/ls /usr/bin/python3 /usr/bin/firefox /bin/bash

2. ПРОЦЕСС

Процесс — это экземпляр программы, который выполняется.
У каждого процесса есть свой PID и ресурсы.



/usr/bin/vim
(программа)

ПРОЦЕСС
(PID: 1234)

- Память
- Файлы
- Устройства
- Сеть

- ✓ Активен (выполняется в системе)
- ✓ Имеет уникальный PID (идентификатор процесса)
- ✓ Получает ресурсы: CPU, память, файлы, устройства ввода-вывода
- ✓ Имеет состояние и атрибуты (пользователь, приоритет и др.)

3. ЖИЗНЕННЫЙ ЦИКЛ ПРОЦЕССА



Создание
(fork/exec)

Процесс создаётся
системой или
пользователем



Выполнение
(running)

Процесс выполняется
с использованием
ресурсов



Ожидание
(waiting)

Процесс ждёт
события или
ресурса



Завершение
(terminated)

Процесс завершает
работу и освобождает
ресурсы



Все процессы рано или поздно завершаются, освобождая ресурсы.

4. ПРОГРАММА vs ПРОЦЕСС

Программа

Файл на диске
Пассивна
Одинаковый файл = одинаковая программа
Не имеет PID
Не использует ресурсы самостоятельно
Пример: /bin/ls

Процесс

Экземпляр программы в памяти
Активен
Один файл может иметь много процессов
Имеет уникальный PID
Использует ресурсы (CPU, память и т.д.)
Пример: ls (PID: 1234)

5. ПРИМЕР: ОДНА ПРОГРАММА — МНОГО ПРОЦЕССОВ

Запущенные процессы

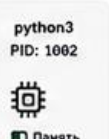


/usr/bin/python3
(программа)



python3
PID: 1001

Память



python3
PID: 1002

Память



python3
PID: 1003

Память

Каждый процесс имеет своё адресное пространство и ресурсы.

6. ПРОСМОТР ПРОЦЕССОВ В LINUX

Основные команды

```
ps aux      # список процессов
ps -ef      # полный формат
ps -u $USER # процессы текущего пользователя
pstree      # дерево процессов
top          # интерактивный монитор
htop         # улучшенный top (если установлен)
```

Пример: ps aux | head -5

USER	PID	%CPU	%MEM	VSZ	RSS	TTY	STAT	START	TIME	COMMAND
user	1001	0.0	0.1	12345	2345	?	Ss	10:00	0:00	bash
user	1234	1.2	2.3	512345	45678	?	Sl	10:01	0:05	firefox
user	2345	0.0	0.2	22345	3456	?	S	10:02	0:00	python3
root	1	0.0	0.1	16200	2340	?	Ss	09:59	0:01	systemd

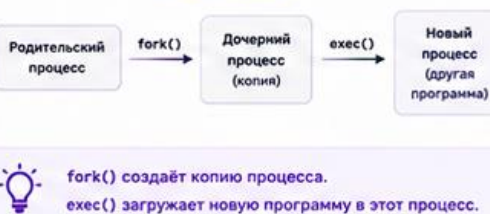
PID — уникальный идентификатор процесса
STAT — состояние процесса (R — running, S — sleeping, Z — zombie и др.)

7. КАК СОЗДАЮТСЯ ПРОЦЕССЫ

Способы создания:

- Запуск из терминала (команда)
- Скрипты и программы
- Системные сервисы (daemon)
- Планировщик задач (cron, systemd timer)
- Другие процессы (fork/exec)

Модель fork/exec



fork() создаёт копию процесса.
exec() загружает новую программу в этот процесс.

8. ПОЛЕЗНО ЗНАТЬ

Идентификаторы

PID — идентификатор процесса
PPID — идентификатор родительского процесса
UID — идентификатор пользователя
GID — идентификатор основной группы

Состояния процессов (STAT)

R — running (выполняется)
S — sleeping (ожидание события)
D — uninterruptible sleep (глубокое ожидание I/O)
T — stopped (остановлен сигналом)
Z — zombie (завершён, но не очищен)

Где хранятся данные о процессах?



/proc
Виртуальная файловая система,
содержащая информацию
о всех процессах.

Например:

```
/proc/1234/ — данные о процессе с PID 1234
/proc/1234/status — статус процесса
/proc/1234/cmdline — команда запуска
```

ЗАЧЕМ ЭТО НУЖНО?

- ✓ Управление системой и ресурсами
- ✓ Диагностика и поиск проблем
- ✓ Настройка безопасности
- ✓ Автоматизация задач
- ✓ Понимание работы Linux



ВАЖНО



Каждый процесс имеет свой PID и ресурсы.



Один и тот же файл программы может иметь много процессов.



Убивайте только те процессы, которые действительно нужны.



Процессы — основа многозадачности Linux.

2. PID, PPID, состояние и владелец

PID — Process ID, уникальный номер процесса.

PPID — Parent Process ID, номер родительского процесса.

Владелец процесса определяет, от чьего имени он работает.

Состояние процесса показывает, выполняется он, спит, завершился или ожидает ресурс.

Посмотреть эти атрибуты процесса можно с помощью команды: **ps -eo pid,ppid,user,stat,comm**

Почему владелец процесса важен

Процесс получает права своего пользователя.

Службы часто запускаются от отдельного системного пользователя.

Если web-служба работает от root без необходимости, это риск.

Команды ps и top помогают увидеть владельца процесса.

3. Просмотр процессов через ps

ps показывает снимок процессов на момент запуска команды.

Пример: **ps aux**

Важные столбцы: USER, PID, %CPU, %MEM, STAT, COMMAND.

Для поиска процесса удобно использовать: **ps aux | grep nginx**

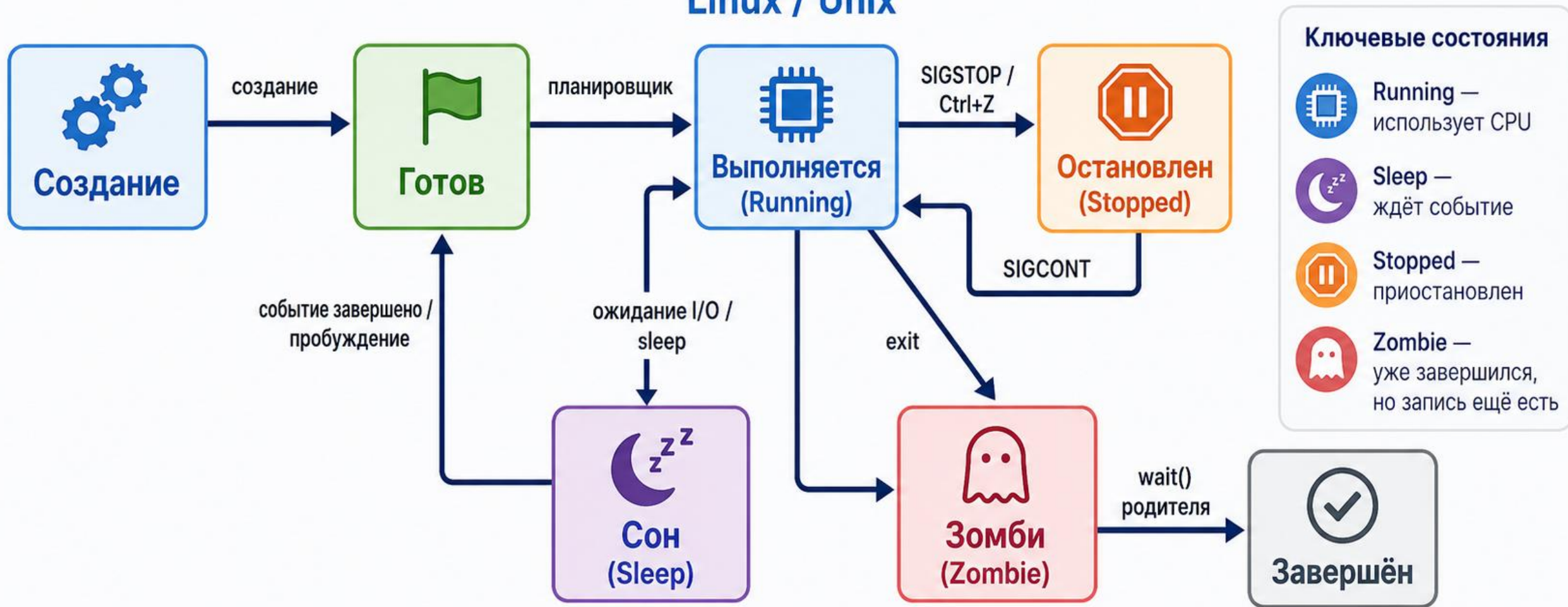
grep может найти и сам себя, поэтому результат нужно читать внимательно.

Шпаргалка по состояниям (столбец STAT / S):

- R (Running) — процесс выполняется прямо сейчас или готов к выполнению (ждет своей очереди к процессору).
- S (Sleeping) — «прерывистый сон». Процесс ожидает какого-то события (сетевое запроса, ввода от пользователя). Это нормальное состояние для 95% процессов сервера.
- D (Uninterruptible Sleep) — «непрерывистый сон». Процесс завис, ожидая ответа от диска или оборудования. Такой процесс нельзя убить даже командой kill -9, пока диск не ответит.
- Z (Zombie) — процесс-зомби. Он завершил работу, но его родитель (PPID) еще не прочитал код завершения. Память не потребляет, но занимает строку в таблице.
- T (Stopped) — процесс остановлен (например, комбинацией Ctrl+Z или сигналом SIGSTOP).
- I (Idle) — (Бездействующий поток ядра)

Блок-схема жизненного цикла процесса

Linux / Unix



Главная идея:

Процесс в ОС постоянно меняет состояние: выполняется, ждёт, останавливается и завершается.

Поиск процессов через **pgrep**

pgrep ищет PID по имени процесса.

Пример: **pgrep -a sshd**

Ключ **-a** показывает не только PID, но и командную строку.

Это удобнее, чем `ps | grep`, когда нужно быстро найти конкретную службу.

top и htop

top показывает процессы в динамике и обновляет экран.

htop удобнее визуально, но может быть не установлен.

В top важны CPU, память, load average и процессы вверху списка.

Для выхода из top нажмите q.

Почему htop — идеальный выбор для старта?

- Наглядность и цветовая индикация: В отличие от классического текстового вывода, htop визуализирует загрузку процессора и оперативной памяти в виде цветных шкал. Сразу видно, если сервер перегружен.
- Все нужные атрибуты на одном экране;
- В htop очень просто показать родительские процессы. Достаточно нажать клавишу F5, и вся таблица превратится в удобное интерактивное «дерево» процессов (Tree view). Наглядно видно, какой процесс кого породил.
- Интерактивное управление: Не нужно запоминать синтаксис команды kill. Можно выбрать зависший процесс стрелочками на клавиатуре, нажать F9 (Kill), выбрать нужный сигнал (SIGTERM или SIGKILL) и завершить его.

4. Сигналы процессам

Сигнал — способ сообщить процессу о событии или требовании.

SIGTERM просит процесс корректно завершиться.

SIGKILL принудительно завершает процесс и не дает ему очистить ресурсы.

SIGHUP часто используется для перечитывания конфигурации.

Администратор сначала пробует мягкие способы, а не сразу SIGKILL.

SIGTERM и SIGKILL

SIGTERM — нормальная просьба завершиться.

Пример: **kill PID**

SIGKILL — жесткое завершение.

Пример: **kill -9 PID**

kill -9 может оставить временные файлы, неполные операции и потерю данных.

5. Завершение процессов

kill отправляет сигнал процессу по PID.

pkill отправляет сигнал процессам по имени или шаблону.

killall завершает процессы по имени команды.

Перед завершением нужно проверить, что выбран именно нужный процесс.

Безопасное завершение процесса

Сначала найдите процесс: `pgrep -a имя`.

Затем попробуйте SIGTERM: `kill PID`.

Проверьте, завершился ли процесс: `pgrep -a имя`.

Только если процесс завис, используйте `kill -9 PID`.

Для служб лучше применять **`systemctl stop`**, если процесс является службой.



СИГНАЛЫ ПРОЦЕССАМ

ЗАВЕРШЕНИЕ ПРОЦЕССОВ. SIGTERM и SIGKILL

Сигнал — это способ уведомить процесс о событии. Процесс может обработать сигнал, проигнорировать его или завершиться.

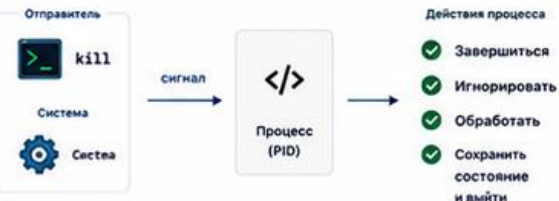
ЗАЧЕМ НУЖНЫ СИГНАЛЫ?



- ☐ Управление процессами
- ☐ Ошибка и диагностика
- ☐ Завершение работы
- ☐ Реакция на события системы
- ☐ Перезапуск сервисов

1. ЧТО ТАКОЕ СИГНАЛ?

Сигнал отправляется процессу (или группе процессов) ядром или пользователем.



Сигналы бывают стандартные и пользовательские.

2. ОСНОВНЫЕ СИГНАЛЫ В LINUX

Сигнал	Номер	Имя	Действие по умолчанию	Описание
1		SIGHUP	Завершение	Завершить процесс (перезапуск при перезагрузке терминала)
2		SIGINT	Завершение	Прерывание с клавиатуры (Ctrl+C)
3		SIGQUIT	Завершение + core	Выход с дампом памяти (Ctrl+\)
9		SIGKILL	Немедленное завершение	Немедленно завершить процесс (не может быть перехвачен)
15		SIGTERM	Завершение	Корректное завершение (по умолчанию kill)
18		SIGCONT	Продолжить	Продолжить остановленный процесс
19		SIGSTOP	Остановить	Немедленно остановить процесс (не может быть перехвачен)

✓ SIGTERM (15) — корректное завершение. SIGKILL (9) — принудительное завершение.

3. SIGTERM И SIGKILL: В ЧЕМ РАЗНИЦА?



SIGTERM (15)

- Мягкое завершение (graceful shutdown)
- Процесс может перехватить (обработать) сигнал
- Позволяет сохранить данные, закрыть соединения, освободить ресурсы
- Программы могут игнорировать или отложить обработку

Пример: `kill 1234` или `kill -15 1234`



SIGKILL (9)

- Жесткое завершение (force kill)
- Нельзя перехватить или проигнорировать
- Процесс убивается немедленно
- Данные могут быть потеряны, ресурсы не освобождены корректно

Пример: `kill -9 1234`

✓ Всегда сначала используйте SIGTERM. Применяйте SIGKILL только если процесс не завершился.

4. КАК ОТПРАВИТЬ СИГНАЛ

Команда	Описание
<code>kill PID</code>	Отправить SIGTERM (15) процессу PID
<code>kill -9 PID</code>	Отправить SIGKILL (9) процессу PID
<code>kill -SIGNAL PID</code>	Отправить указанный сигнал процессу
<code>kill -l</code>	Показать список сигналов
<code>pkill имя</code>	Отправить SIGTERM процессам по имени
<code>pkill -9 имя</code>	Отправить SIGKILL процессам по имени

💡 Без параметров `kill` отправляет сигнал SIGTERM (15).

5. ПРИМЕРЫ ИСПОЛЬЗОВАНИЯ

Завершение процесса по PID

```
$ ps aux | grep nginx
user 1234 0.0 0.1 12345 2345 ? Ss 10:00 0:00 nginx: master
$ kill 1234 # SIGTERM (мягкое завершение)
$ kill -9 1234 # SIGKILL (принудительное)
```

Остановка и продолжение

```
$ sleep 1000 &
[1] 5678
$ kill -STOP 5678 # остановить (приостановить)
$ kill -CONT 5678 # продолжить
```

Завершение по имени процесса

```
$ pkill firefox # SIGTERM всем процессам firefox
$ pkill -9 firefox # SIGKILL всем процессам firefox
```

❗ SIGSTOP и SIGKILL не могут быть перехвачены, остальные сигналы — могут.

6. КАК УЗНАТЬ ТЕКУЩЕЕ СОСТОЯНИЕ ПРОЦЕССА

Команда	Описание
<code>ps aux</code>	Показать все процессы
<code>ps -o pid,ppid,stat,cmd -p PID</code>	Подробная информация о процессе
<code>top / htop</code>	Интерактивный мониторинг
<code>jobs</code>	Список фоновых заданий в текущем shell

Состояния (STAT):

R - running	(выполняется)	Z - zombie	(процесс-зомби)
S - sleeping	(ожидает событие)	D - uninterruptible sleep	(непрерываемый сон)
T - stopped	(остановлен сигналом)		

7. ПОРЯДОК ЗАВЕРШЕНИЯ ПРОЦЕССА

- 1 Пробуем SIGTERM
Отправляем сигнал корректного завершения
`$ kill PID`
- 2 Ждем завершения
Даем процессу время на корректное завершение (несколько секунд)
- 3 Проверяем
Проверяем, жив ли процесс
- 4 Если не завершился — SIGKILL
Принудительно убиваем процесс
`$ kill -9 PID`

Процесс
завершен
Ресурсы
освобождены

8. ПРИМЕР: ЗАВЕРШЕНИЕ ЗАВИСЯЩЕГО ПРОЦЕССА

```
$ ps aux | grep myapp
user 4321 0.0 0.1 23456 3456 ? Ss 11:00 0:00 myapp
$ kill 4321 # пробуем мягко
# процесс не завершился...
$ kill -9 4321 # принудительно
$ ps aux | grep myapp
# (ничего не выводится)
# процесс завершен
```

9. ПОЛЕЗНЫЕ СОВЕТЫ

- ✓ Всегда сначала используйте SIGTERM (`kill PID`).
- ✓ Используйте SIGKILL только в крайних случаях.
- ✓ Для сервисов используйте системные команды: `systemctl stop <service>`
- ✓ Логируйте и обрабатывайте сигналы в своих скриптах (`trap` в `bash`, `signal()` в `C` и др.).



Грамотное использование сигналов делает системы стабильнее и безопаснее.



ЗАПОМНИТЬ



`kill PID` → SIGTERM (15)
мягкое завершение



`kill -9 PID` → SIGKILL (9)
жесткое завершение



Не все сигналы можно перехватить:
SIGKILL и SIGSTOP — нельзя.



Сигналы можно отправлять
процессам и их группам.



Правильное завершение = сохранность
данных и освобождение ресурсов.

6. Стандартные потоки

stdin — стандартный ввод, откуда программа получает данные.

stdout — стандартный вывод, куда программа пишет обычный результат.

stderr — стандартный поток ошибок.

Разделение `stdout` и `stderr` помогает сохранять результат и ошибки отдельно.

Пример stdout и stderr

Команда **ls /etc** выводит список в stdout.

Команда **ls /no_such_dir** пишет ошибку в stderr.

Оба потока обычно видны на экране, но перенаправляются разными операторами.

Это важно для скриптов и логирования.

7. Перенаправление вывода

> записывает stdout в файл с перезаписью.

Пример: **ls /etc > etc-list.txt**

>> добавляет stdout в конец файла.

Пример: **date >> report.log**

Перед использованием > проверьте, не перезаписываете ли важный файл.

Перенаправление ошибок

2> записывает stderr в файл.

Пример: **ls /no_such_dir 2> errors.log**

2>&1 объединяет stderr со stdout.

Пример: **команда > all.log 2>&1**

Такой прием часто применяют в cron и скриптах.



СТАНДАРТНЫЕ ПОТОКИ: **stdin**, **stdout**, **stderr**

ПЕРЕНАПРАВЛЕНИЕ ВЫВОДА В LINUX

Каждый процесс в Linux по умолчанию имеет три стандартных потока ввода-вывода.



Всё в Linux — это файлы.

Стандартные потоки — это специальные файловые дескрипторы:

0 — **stdin** (ввод) 1 — **stdout** (вывод) 2 — **stderr** (ошибки)

1. ЧТО ТАКОЕ СТАНДАРТНЫЕ ПОТОКИ?

Каждый процесс имеет три стандартных потока:

0

stdin

(standard input)



Стандартный ввод
Данные, которые читает программа.
По умолчанию: клавиатура

1

stdout

(standard output)



Стандартный вывод
Результаты работы программы.
По умолчанию: экран (терминал)

2

stderr

(standard error)



Стандартный поток ошибок
Сообщения об ошибках.
По умолчанию: экран (терминал)

2. КАК РАБОТАЮТ ПОТОКИ?

Программа общается с внешним миром через эти три канала.

Источники ввода



Клавиатура



Файл / pipe

...

0 **stdin**

ПРОЦЕСС
(команда, программа)



1 **stdout**

2 **stderr**

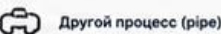
Назначения вывода



Экран (терминал)



Файл



Другой процесс (pipe)

...



stdout → для нормального вывода
stderr → для сообщений об ошибках
Их можно перенаправлять независимо!

3. ПРОСМОТР ПОТОКОВ ПРОЦЕССА

Команда `ls` с разными типами вывода:

```
$ ls /no_such_dir
ls: cannot access '/no_such_dir': No such file or directory
$ echo "Привет"
Привет
```

`echo "Привет"` → **stdout** (1)

Сообщение об ошибке `ls` → **stderr** (2)

4. ПЕРЕНАПРАВЛЕНИЕ ВЫВОДА

> Перенаправление stdout в файл

Перезаписать файл данными из **stdout**.

```
$ echo "Привет" > out.txt
$ cat out.txt
Привет
```



>> Добавление stdout в файл

Добавить данные в конец файла.

```
$ echo "Строка 1" > out.txt
$ echo "Строка 2" >> out.txt
$ cat out.txt
Строка 1
Строка 2
```



2> Перенаправление stderr в файл

Перезаписать файл данными из **stderr**.

```
$ ls /no_such_dir 2> err.txt
$ cat err.txt
ls: cannot access '/no_such_dir':
No such file or directory
```



&> Перенаправление stdout и stderr

Оба потока в один файл (перезапись).

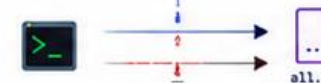
```
$ ls /no_such_dir &> all.log
$ cat all.log
ls: cannot access '/no_such_dir':
No such file or directory
```



>>& Добавление stdout и stderr в файл

Оба потока в один файл (добавление).

```
$ ls /no_such_dir >>& all.log
$ cat all.log
ls: cannot access '/no_such_dir':
No such file or directory
```



>/dev/null Отбросить вывод

Отбросить **stdout** (или **stderr**).

```
$ command > /dev/null
$ command 2> /dev/null
$ command > /dev/null 2>&1
```

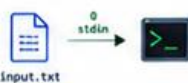


5. ПЕРЕНАПРАВЛЕНИЕ ВВОДА (stdin)

< Перенаправить входные данные из файла.

```
$ cat < input.txt
$ sort < data.txt
```

Пример:



6. ПОЛЕЗНЫЕ КОМБИНАЦИИ

Команда

`command > file 2>&1`

`command >> file 2>&1`

`command 2>&1 > file`

`command > file`

`command 2> file`

`command < input.txt`

Что делает

stdout и **stderr** в один файл (перезапись)

stdout и **stderr** в один файл (добавление)

Сначала объединяет потоки, потом перенаправляет **stdout** (НЕПРАВИЛЬНО)

Только **stdout** в файл

Только **stderr** в файл

stdin из файла



Порядок перенаправлений важен!

`2>&1 > file #`
`> file 2>&1`

7. ПРИМЕРЫ ИЗ ПРАКТИКИ

```
$ make > build.log 2>&1 # Сохраним весь вывод сборки
$ command >> out.txt 2> err.txt
# Раздельно: stdout в out.txt, stderr в err.txt
$ grep "error" app.log 2>/dev/null # Только stdout, ошибки отбросить
$ some_command &> /dev/null # Отбросить весь вывод
$ python script.py > out.txt 2> err.txt
# Раздельно результаты и ошибки
```

8. КРАТКО

0 **stdin** — ввод из клавиатуры или файла
1 **stdout** — нормальный вывод
2 **stderr** — сообщения об ошибках
> перезаписать файл
>> добавить в конец файла
2> ошибки в файл
&> **stdout** и **stderr** в один файл
>/dev/null отбросить вывод



ЗАПОМНИТЬ



stdout (1) — для результата
stderr (2) — для ошибок



Их можно перенаправлять независимо друг от друга



Порядок перенаправлений может менять результат



Храни логи в файлах — это упрощает отладку



Команда `man bash` → раздел **REDIRECTION** для подробностей

8. Pipeline

Pipeline или конвейер передает stdout одной команды на stdin другой.

Символ конвейера: |

Пример: **ps aux | grep ssh**

Каждая команда в конвейере должна выполнять небольшую понятную задачу.

Конвейеры позволяют строить диагностику из простых инструментов.

Пример анализа файла через pipeline

Нужно найти уникальные имена пользователей из файла.

Пример: **cut -d: -f1 /etc/passwd | sort | uniq**

cut берет первый столбец, sort сортирует, uniq убирает повторы.

Такой подход показывает силу Unix-философии: маленькие инструменты вместе.

9. grep

grep фильтрует строки по шаблону.

Пример: **grep '^root' /etc/passwd**

Ключ **-i** игнорирует регистр.

Ключ **-n** показывает номера строк.

grep полезен для журналов, конфигураций и вывода других команд.

Базовые утилиты обработки текста и конвейеры

В Linux утилиты `cut`, `sort`, `uniq` и `wc` являются мощными инструментами для анализа логов, конфигураций и списков.

Чаще всего они используются не по отдельности, а связываются друг с другом с помощью конвейера (`|`), где вывод одной команды мгновенно передается на вход следующей.

Утилита cut (Нарезка строк на столбцы)

Используется для того, чтобы «вырезать» из текстовых строк только нужные столбцы (поля) или символы.

Основные флаги:

-d (delimiter) — задает символ-разделитель столбцов (по умолчанию это знак табуляции).

-f (fields) — указывает номера столбцов, которые нужно оставить.

Примеры использования:

Вырезать только имена пользователей (1-й столбец) из файла /etc/passwd, где разделителем является двоеточие ":"

```
cut -d ':' -f 1 /etc/passwd
```

Вырезать 1-й и 7-й столбцы (имя и командную оболочку)

```
cut -d ':' -f 1,7 /etc/passwd
```

Утилита `sort` (Сортировка строк)

Используется для упорядочивания строк по алфавиту или по числам. Без предварительной сортировки многие другие утилиты (например, `uniq`) работают некорректно.

Основные флаги:

- n (numeric) — сортировать как числа, а не по алфавиту (иначе число 10 встанет выше, чем 2).
- r (reverse) — развернуть сортировку (от большего к меньшему, от Я до А).
- t (field-separator) — задать разделитель полей, если нужно сортировать по конкретному столбцу.
- k (key) — указать номер столбца для сортировки.

Примеры использования:

Отсортировать список пользователей по алфавиту

```
cut -d ':' -f 1 /etc/passwd | sort
```

Отсортировать процессы по PID (численная сортировка от большего к меньшему)

```
ps -eo pid,comm | sort -nk 1 -r
```

Утилита `uniq` (Фильтрация повторяющихся строк)

Удаляет последовательно идущие дубликаты строк.

Критически важное правило: `uniq` «видит» дубликаты только если они идут друг за другом. Поэтому перед применением `uniq` текст всегда нужно обрабатывать командой `sort`.

Основные флаги:

- c (`count`) — подсчитать, сколько раз встретилась каждая уникальная строка.

- u (`unique`) — показать только те строки, которые вообще не имели дубликатов.

- d (`repeated`) — показать только те строки, которые дублировались.

Пример использования:

Найти все уникальные командные оболочки, назначенные пользователям

```
cut -d ':' -f 7 /etc/passwd | sort | uniq
```


Утилита **wc** (Word Count — Подсчет статистики текста)

Используется для подсчета количества строк, слов и байт (символов) в файле или в потоке данных.

Основные флаги:

-l (lines) — подсчитать только количество строк. (Самый частый сценарий для сисадмина, чтобы узнать число ошибок в логе или количество пользователей).

-w (words) — подсчитать количество слов.

-c (bytes/characters) — подсчитать количество байт/символов.

Примеры использования:

Посчитать, сколько всего учетных записей зарегистрировано в системе

```
wc -l /etc/passwd
```

Узнать, сколько строк лога содержат ошибку "error"

```
grep "error" /var/log/syslog | wc -l
```

Реальный пример комплексной диагностики (Pipeline)

Представьте, что веб-сервер Nginx атаковали, и администратору нужно срочно составить ТОП-5 IP-адресов, которые чаще всего обращались к серверу, согласно файлу логов (access.log), где IP-адрес идет самым первым полем.

Команда в одну строчку будет выглядеть так:

```
cat access.log | cut -d ' ' -f 1 | sort | uniq -c | sort -nk 1 -r | head -n 5
```

Разбор работы конвейера:

- `cat access.log` — читает лог-файл и отправляет текст дальше.
- `cut -d ' ' -f 1` — вырезает из каждой строки только первый столбец (IP-адрес), отбрасывая дату, время и запросы.
- `sort` — группирует одинаковые IP-адреса друг за другом.
- `uniq -c` — схлопывает дубликаты и приписывает к каждому IP-адресу количество его упоминаний (например: 1500 192.168.1.50).
- `sort -nk 1 -r` — сортирует полученный список по первому столбцу (числу запросов) в обратном порядке (от самых активных к менее активным).
- `head -n 5` — оставляет на экране только первые 5 строк (самых главных виновников нагрузки).



БАЗОВЫЕ УТИЛИТЫ ОБРАБОТКИ ТЕКСТА: cut, sort, uniq и wc И КОНВЕЙЕРЫ

Простые инструменты — мощные возможности. Комбинируйте утилиты с помощью конвейеров (|).



Конвейер (|) передаёт вывод одной команды на вход следующей.

cmd1 | cmd2 | cmd3

1. cut — ВЫРЕЗАЕТ ЧАСТИ СТРОК

Позволяет вырезать колонки (поля) или символы из каждой строки.

Файл: data.txt



```
"id;name;age;city"
1;Alice;25;Almaty
2;Bob;30;Astana
3;Carol;22;Shymkent"
```

Примеры

```
$ Вырезать 2-ю колонку (имя)
$ cut -d ';' -f 2 data.txt
Alice
Bob
Carol

# Вырезать колонки 1 и 3
$ cut -d ';' -f 1,3 data.txt
id;age
1;25
2;30
3;22

# Вырезать символы 2-5
$ cut -c 2-5 data.txt
id;;n
;Ali
;Bob
;Car
```

Ключи cut

-d <символ> — разделитель полей (по умолчанию TAB)

-f <список> — выбрать поля (колонки)

-c <список> — выбрать символы по позициям

--complement — инвертировать выбор

2. sort — СОРТИРУЕТ СТРОКИ

Сортирует строки текста по алфавиту, числовому значению или другим критериям.

Примеры

```
$ cat names.txt
Bob
Alice
Carol
Dave

# Сортировка по алфавиту
$ sort names.txt
Alice
Bob
Carol
Dave

# Обратная сортировка
$ sort -r names.txt
Dave
Carol
Bob
Alice

# Числовая сортировка
$ sort -n numbers.txt
2
10
25
100
```

Ключи sort

-n — числовая сортировка

-r — обратный порядок

-k N — сортировка по N-му полю

-d — по словарю, посимвольно

-u — только уникальные строки

-t X — разделитель полей (по умолчанию пробел)



По умолчанию sort чувствителен к регистру. Для игнорирования регистра используйте:

```
$ sort -f names.txt
```

3. uniq — УДАЛЯЕТ (ИЛИ ПОКАЗЫВАЕТ) ДУБЛИКАТЫ

Удаляет повторяющиеся соседние строки (обычно используется после sort).

Примеры

```
$ cat items.txt
apple
banana
apple
orange
banana
banana
pear

# Удалить дубликаты (только соседние!)
$ sort items.txt | uniq
apple
banana
orange
pear

# Показать только дубликаты
$ sort items.txt | uniq -d
apple
apple
banana

# Показать, сколько раз встречается
$ sort items.txt | uniq -c
2 apple
3 banana
1 orange
1 pear
```

Ключи uniq

-c — вывести количество повторений

-d — вывести только дубликаты

-u — вывести только уникальные строки

-i — не учитывать регистр



Важно! uniq работает только с соседними одинаковыми строками. Сначала сортируйте!

4. wc — СЧИТАЕТ СТРОКИ, СЛОВА И БАЙТЫ

Подсчитывает количество строк, слов, символов и байтов в файле или потоке.

Примеры

```
$ cat file.txt
Hello world
This is Linux
Text processing is fun

$ wc file.txt
3 8 34 file.txt
# строки слова байты

$ wc -l file.txt # только строки
3 file.txt

$ wc -w file.txt # только слова
8 file.txt

$ wc -c file.txt # только байты
34 file.txt

$ wc -m file.txt # символы (UTF-8)
33 file.txt
```

Ключи wc

-l — количество строк

-w — количество слов

-c — количество байтов

-m — количество символов

-L — длина самой длинной строки



Слова — это последовательности символов, разделённые пробелами, табами или переводами строк.

5. КОНВЕЙЕРЫ (|) — СОЕДИНЯЕМ КОМАНДЫ

Как работает конвейер?

Вывод (stdout) одной команды становится входом (stdin) для следующей.



Преимущества:

- ✓ Меньше временных файлов
- ✓ Быстрее и эффективнее
- ✓ Гибкое комбинирование утилит

Задача

Команда

Отсортировать строки файла

```
$ sort file.txt
```

Удалить дубликаты из файла

```
$ sort file.txt | uniq
```

Посчитать уникальные слова

```
$ tr '[:upper:]' '[:lower:]' < file.txt |
tr -cs 'a-z-a-z0-9' '\n' | sort | uniq -c | sort -nr
```

Показать 5 самых частых слов

```
$ tr -cs 'a-z-a-z0-9' '\n' < file.txt |
sort | uniq -c | sort -nr | head -5
```

Вырезать 2-й столбец и отсортировать

```
$ cut -d ';' -f 2 data.txt | sort
```

Посчитать строки с ошибками в логах

```
$ grep 'ERROR' app.log | wc -l
```

Посчитать количество уникальных IP в логе

```
$ cut -d ' ' -f 1 access.log | sort | uniq | wc -l
```

Сортировка по алфавиту

Сначала сортируем, потом убираем повторы

Нормализуем регистр, выделяем слова, считаем частоту

Топ-5 самых частых слов

Получаем 2-й столбец и сортируем

Фильтруем и считаем строки

Количество уникальных IP-адресов

Пример: анализ лог-файла

```
$ cat access.log
192.168.1.1 - - [20/May/2024] "GET /index.html"
192.168.1.2 - - [20/May/2024] "GET /about.html"
192.168.1.1 - - [20/May/2024] "GET /contact.html"
192.168.1.3 - - [20/May/2024] "GET /index.html"
192.168.1.2 - - [20/May/2024] "GET /index.html"

# Уникальные IP и количество запросов
$ cut -d ' ' -f 1 access.log | sort | uniq -c | sort -nr
2 192.168.1.1
2 192.168.1.2
1 192.168.1.3
```

Полезные связки

```
$ grep 'error' file.log | sort | uniq -c | sort -nr
$ cat file.txt | cut -c 1-10 | sort | uniq
$ find . -type f | wc -l
$ ps aux | grep nginx | wc -l
$ netstat -tuln | grep LISTEN | wc -l
```



Совет

Комбинируйте утилиты — это основа работы в Linux! Освойте пайпы — и вы сможете решать сложные задачи в пару команд.



БЫСТРАЯ
СПРАВКА

cut

-d ';' -f N # поле N

-c N-M # символы N-M

sort

-n # числовая

-r # обратная

-u # уникальные

uniq

-c # количество

-d # только дубликаты

-u # только уникальные

wc

-l # строки

-w # слова

-c # байты

-m # символы



+ cmd1 | cmd2 + cmd3

Вывод одной команды становится входом другой.



ВАЖНО

Сначала сортируйте (sort), потом удаляйте дубликаты (uniq)!



ПРАКТИКУЙТЕСЬ!

Лучший способ запомнить — пробовать на реальных данных.

Почему процессы нужно понимать до настройки служб

- Любая служба в Linux работает как один или несколько процессов.
- Если процесс завис, служба может считаться запущенной, но фактически не работать.
- Нагрузка CPU, RAM и диска часто видна через процессы.
- Управление процессами помогает аккуратно остановить проблему без перезагрузки сервера.

Фоновые задания в shell

В операционной системе Linux администратор может выполнять длительные задачи (например, архивацию больших каталогов, копирование терабайтных бэкапов или сетевой мониторинг), не блокируя при этом терминал.

Механизм управления задачами (Job Control) позволяет переключать процессы между активным (передним) и фоновым режимами.

Запуск команды в фоне: Символ &

Если в конце любой команды поставить амперсанд (&), оболочка Shell мгновенно запустит этот процесс в фоновом режиме (background) и сразу же вернет управление пользователю для ввода новых команд.

Синтаксис: **команда &**

Пример применения:

tar -czf backup.tar.gz /var/www/html &

Что выведет терминал:

Система выдаст сообщение вида [1] 28450, где:

- [1] — номер задания (job ID) внутри текущей сессии Shell (уникален только для текущего окна терминала).
- 28450 — общесистемный PID процесса.

Просмотр фоновых задач: Команда jobs

Используется для вывода списка всех фоновых и приостановленных задач, которые были запущены именно в текущем сеансе связи (терминале).

Синтаксис: **jobs [опции]**

Популярные флаги:

- -l — дополнительно выводить системный PID для каждого задания.

Пример вывода:

```
[1]- Running      ping 8.8.8.8 > /dev/null &  
[2]+ Stopped      nano config.txt
```

- Знак + показывает задачу, к которой команды fg или bg обратятся по умолчанию (последнее измененное задание).
- Знак - показывает предпоследнюю задачу.
- Running / Stopped — текущий статус процесса в фоне.

Возврат на передний план: Команда **fg** (Foreground)

Переводит выбранное фоновое или приостановленное задание на передний план терминала (foreground). Процесс снова забирает фокус ввода, и вы не сможете вводить другие команды, пока он не завершится.

Синтаксис: **fg [%номер_задания]**

Примеры использования:

- **fg** # Вернет на передний план задачу со знаком "+" (в примере выше — nano)
- **fg %1** # Принудительно вернет задачу №1 (в примере выше — ping)

Отправка запущенной задачи в фон «на ходу»: Ctrl+Z и bg

Бывают ситуации, когда администратор запустил тяжелую утилиту (например, `sr -r`), забыл поставить `&`, и терминал оказался заблокирован.

Механизм фонового контроля позволяет исправить это без прерывания работы утилиты.

Пошаговый алгоритм:

1. Прямо во время выполнения команды нажмите комбинацию клавиш `Ctrl+Z`.

Что произойдет: Процесс мгновенно заморозится (ядро отправит ему сигнал `SIGSTOP`). В терминале появится статус `Stopped`, а управление вернется сисадмину. Но в этот момент процесс не работает, он «поставлен на паузу».

2. Чтобы заставить его продолжить работу, но уже скрытно в фоновом режиме, введите команду `bg` (Background):

`bg [%номер_задания]`

Пример: **`bg %1`** переведет замороженную задачу в статус `Running` внутри фона.

Шпаргалка для администратора по Job Control

Действие в терминале	Результат для процесса
команда &	Сразу запускает задачу скрытно в фоне.
Ctrl+C	Полностью убивает (SIGINT) текущий интерактивный процесс.
Ctrl+Z	Не убивает, а временно замораживает (Stopped) процесс.
jobs	Показывает шпаргалку со списком всех фоновых задач текущего окна.
fg %Номер	Забирает процесс из фона обратно на экран.
bg %Номер	Будит замороженный процесс и заставляет его работать в фоне.



ФОНОВЫЕ ЗАДАНИЯ В SHELL

Запускайте команды в фоновом режиме и управляйте ими.



Фоновое задание (job) — команда, запущенная в фоновом режиме. Shell не ждёт её завершения и возвращает управление вам.

1. ЗАПУСК В ФОНОВОМ РЕЖИМЕ

Добавьте `&` в конец команды — она будет выполнена в фоне.

```
$ long_task.sh &
[1] 12345
$ sleep 100 &
[2] 12346
$ echo "Работаю дальше..."
Работаю дальше...
```

[1] — номер задания (job ID)
12345 — PID процесса

✓ Вы сразу возвращаетесь к приглашению оболочки.



Любую команду можно запустить в фоне: команды, скрипты, программы.

2. ПРОСМОТР ФОНОВЫХ ЗАДАНИЙ

Команда `jobs` показывает список фоновых (и остановленных) заданий текущей оболочки.

```
$ jobs
[1]  Running  long_task.sh &
[2]  Running  sleep 100 &
```

Состояния

Running — задание выполняется
Stopped — задание остановлено (например, `Ctrl+Z`)
Done — задание завершено



`jobs -l` — показать PID каждого задания.

3. УПРАВЛЕНИЕ ЗАДАНИЯМИ

`fg %n` — перевести задание `n` в передний план
`bg %n` — продолжить выполнение остановленного задания в фоне
`kill %n` — завершить задание `n`
`kill -SIGNAL %n` — отправить сигнал заданию

```
$ fg %1 # вернуть задание 1 в передний план
long_task.sh
$ bg %1 # продолжить в фоне
[1] long_task.sh &
$ kill %2 # завершить задание 2
```

✓ `%n` — номер задания из списка `jobs`.

4. ОСТАНОВКА И ПРОДОЛЖЕНИЕ (`Ctrl+Z` / `bg` / `fg`)

Нажмите `Ctrl+Z`, чтобы приостановить задание в переднем плане.

```
$ ./count.sh
1
2
3
^Z
[1]+  Stopped                  ./count.sh
```

Команды для управления:

```
$ bg %1 # продолжить выполнение в фоне
[1]+  ./count.sh &
$ fg %1 # вернуть в передний план
./count.sh
```



`Ctrl+C` — завершает задание в переднем плане.
`Ctrl+Z` — приостанавливает (останавливает, не завершает).

5. ПОЛЕЗНЫЕ КОМАНДЫ

<code>jobs</code>	показать список заданий
<code>jobs -l</code>	показать PID и подробности
<code>fg %n</code>	перевести задание <code>n</code> в передний план
<code>bg %n</code>	продолжить задание <code>n</code> в фоне
<code>kill %n</code>	завершить задание <code>n</code> (<code>SIGTERM</code>)
<code>kill -9 %n</code>	принудительно завершить (<code>SIGKILL</code>)
<code>disown %n</code>	удалить задание из списка <code>jobs</code> (shell перестанет его отслеживать)
<code>nohup cmd &</code>	запустить в фоне, игнорируя выход из shell



`disown` — полезно, если хотите сохранить задание после выхода из оболочки без использования `nohup`.

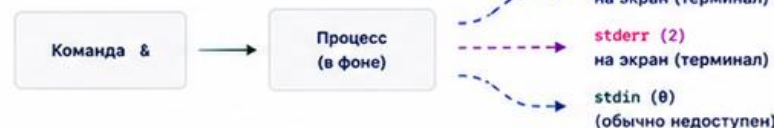
6. ПРИМЕР РАБОЧЕГО ПРОЦЕССА

1. Запуск в фоне
2. Смотрим список
3. Останавливаем
4. Продолжаем в фоне
5. Возвращаем в передний план
6. Завершаем

```
$ find / -type f > all_files.txt &
[1] 22345
$ jobs
[1]  Running  find / -type f > all_files.txt &
$ fg %1 # выводим на передний план
^Z
[1]+  Stopped                  find / -type f > all_files.txt
$ bg %1 # продолжаем в фоне
[1]+  find / -type f > all_files.txt &
$ jobs -l
[1]+  22345 Running          find / -type f > all_files.txt
$ kill %1 # корректное завершение
[1]+  Terminated            find / -type f > all_files.txt
```

7. ПОТОК ВЫВОДА И ФОН (ВАЖНО!)

При запуске в фоне вывод по-прежнему идёт в терминал.



Чтобы перенаправить вывод фонового задания в файл:

```
$ long_task.sh > out.log 2>&1 & # stdout и stderr в один файл
$ long_task.sh >> out.log & # дописать в файл
$ long_task.sh >> out.log 2> err.log & # разные файлы
```



После выхода из shell обычные фоновые задания будут завершены. Используйте `nohup` или `disown` для их сохранения.



ЗАПОМНИТЬ

- ✓ `&` — запустить в фоне
- ✓ `Ctrl+Z` — приостановить
- ✓ `jobs` — список заданий

СИГНАЛЫ (kill)

`SIGTERM` (15) — корректное завершение (kill)
`SIGKILL` (9) — принудительное завершение (kill -9)
`SIGHUP` (1) — завершение при выходе из shell (обычно)



Долгие задачи
Запускайте в фоне и продолжайте работу.



ПОЛЕЗНЫЕ СЦЕНАРИИ

Лог-файлы
Перенаправляйте вывод в файлы.



Серверы и сервисы
Используйте `nohup` или `disown` для надёжности.



ВАЖНО

Будьте аккуратны с `kill -9` — процесс может не успеть сохранить данные.

Сигналы процессов

Сигнал TERM просит процесс завершиться штатно.

Сигнал KILL принудительно завершает процесс и не дает ему обработать завершение.

Сначала используют обычное завершение, и только затем принудительное.

Команда kill отправляет сигнал процессу по PID.

Типовые ошибки

- Использовать `kill -9` без попытки корректного завершения.
- Перезаписать файл через `>` вместо добавления через `>>`.
- Не отделить `stderr` от `stdout` в скрипте.
- Построить длинный `pipeline` и не понимать, какой этап дал ошибку.
- Завершать процесс службы напрямую вместо `systemctl`.

Алгоритм действий администратора при аварии

Когда сервер «тормозит», сайты не открываются, а службы выдают ошибки, администратор должен определить, какой именно процесс виновен в сбое.

Диагностика строится вокруг трех главных дефицитных ресурсов сервера: Процессор (CPU), Оперативная память (RAM) и Дисковый ввод-вывод (I/O).

Администратор действует по следующему чек-листу, связывая изученные инструменты:

Шаг 1: Оценка общей картины.

Запускает `top` (или `htop`). Смотрит на Load Average и показатели памяти.

Шаг 2: Локализация виновника.

Ищет в списке конкретный процесс, который забирает максимум ресурсов (%CPU или %MEM). Запоминает его числовой идентификатор — PID.

Шаг 3: Проверка деталей.

Если имя процесса обрезано, запускает `ps -fp [PID]`, чтобы увидеть, какой именно скрипт или команда сгенерировали этот процесс.

Шаг 4: Попытка мягкого решения.

Отправляет процессу штатный сигнал на завершение:

`sudo kill -15 [PID] # (Отправка SIGTERM)`

Это позволяет программе корректно закрыть файлы, разорвать сетевые соединения и не повредить базу данных.

Шаг 5: Жесткое решение (если шаг 4 не помог).

Если процесс завис намертво и не реагирует на SIGTERM, применяется принудительное уничтожение:

`sudo kill -9 [PID] # (Отправка SIGKILL)`

Ядро Linux мгновенно выкидывает процесс из оперативной памяти, освобождая ресурсы сервера.

Шаг 6: Финальная проверка.

Повторно запускается tor или ps, чтобы убедиться, что нагрузка на сервер упала, а служба (если это был дочерний процесс) успешно перезапустилась

Итоги лекции

- Команды Linux нужно понимать через объект, с которым они работают.
- Любое изменение системы завершается проверкой результата.
- Опасные команды требуют предварительной проверки пути, прав и текущего пользователя.
- Отчет по лабораторной должен содержать не только команды, но и выводы по результатам.

Контрольные вопросы

1. Чем процесс отличается от программы?
2. Что такое PID и PPID?
3. Чем SIGTERM отличается от SIGKILL?
4. Для чего нужны stdout и stderr?
5. Как работает pipeline?